

Daniel Strick
DS542
Midterm Report

AI Disclosure

How I Used AI

I utilized AI assistance extensively throughout this project to deepen my understanding of deep learning, debug my code, and explore strategies to improve my model's performance. My goal was to use AI as a mentor to guide my learning, not to complete the work for me. I primarily used the following AI tools:

- **ChatGPT:** For general questions about deep learning concepts, debugging, and suggestions on model improvement. It was the most helpful tool, but due to free version limitations, I couldn't maintain a single conversation thread, which sometimes hindered its ability to understand the full context of my project.
- **Grok:** To ask questions and seek debugging help, though I found its server response times slow, limiting its usefulness.
- **Claude:** For code-related assistance, as it's known for its coding capabilities. However, the free version's message limits restricted its use.
- **Gemini:** I didn't intentionally use Gemini, but it provided answers automatically during Google searches, which I occasionally referenced for deep learning literature and concepts.

I did my best not to copy and paste code directly from AI into my scripts, focusing instead on understanding and implementing the concepts myself. However, after struggling with implementing early stopping, I used code provided by ChatGPT and Claude, as noted in the codebase comments (### Early Stopping -- code from Claude + ChatGPT).

Parts Written by Me vs. AI Assistance

- **Written by Me:**
 - Initial setup of the SimpleCNN architecture, including the convolutional layers, ReLU activations, max-pooling, and fully connected layer.
 - Data loading and transformation setup for CIFAR-100, including the 80/20 train/validation split and DataLoader configuration.
 - Experimentation with different models (ResNet-18/50) and hyperparameter tuning using WandB.
 - Data augmentation strategy, including ablation studies to test combinations like RandomHorizontalFlip and resizing.
 - Git-based experiment tracking to save model versions.
- **Written with AI Assistance:**

- The train and validate functions were initially written by me but refined with AI suggestions for better structure
- The EarlyStopping class was directly sourced from ChatGPT and Claude after my attempts failed, as noted in the code comments.
- Suggestions for switching to EfficientNet-V2-L, or ResNet and resizing CIFAR-100 images were based on AI recommendations after ResNet failed.
- Used ChatGPT to format the report and organize my thoughts for the write up.

Model Description

Chosen Architecture: ResNet-50 with IMAGENET1K_V1 Weights

I started by writing my own SimpleCNN to learn the basics of neural networks, but it plateaued at around 38% CIFAR-100 test accuracy. Then I explored various pre-defined and pre-trained models to leverage transfer learning, starting with ResNet-18 and ResNet-50, which provided significant improvements in performance.

Architecture Overview:

ResNet-50, part of the ResNet family, is a deep residual network designed to avoid the vanishing gradient problem using skip connections. It consists of:

- 50 layers with a combination of convolutional layers, batch normalization, and ReLU activations.
- Residual connections that allow the model to learn identity functions, helping to avoid degradation in performance as the depth of the network increases.

Justification for Choosing ResNet-50:

- **ResNet's strength** lies in its residual connections. These connections allow the model to use very deep networks to train effectively. It is known to perform well on a wide range of image recognition tasks and commonly is used in transfer learning.
- **Transfer Learning:** Using ResNet-50 pre-trained on ImageNet allowed the model to leverage learned features (edges, textures). It then fine-tuned for the CIFAR-100 dataset, improving performance compared to simpler architectures like SimpleCNN.

Hyperparameter Tuning

Hyperparameter Search Process: I used WandB to track experiments and manually searched for optimal hyperparameters:

- **Epochs:** Initially set to 5 epochs for SimpleCNN, but reduced to 4 epochs for ResNet-50, implementing early stopping (patience=3). My most sophisticated model with data augmentation

only made it 10 epochs before experiencing a plateau.

- **Learning Rate:** Starting at 0.1 caused unstable training for SimpleCNN. After experimenting, I reduced the learning rate to 0.00007 for ResNet-50 to fine-tune pre-trained weights without disrupting them.
- **Batch Size:** I started with 8 for SimpleCNN, and increased it to 128 for ResNet-50 to better utilize the GPU. I derived the number 128 by using the provided SEARCH_BATCH_SIZES code.
- **Optimizer:** Adam was chosen due to its adaptive learning rate capabilities, which is effective for deep networks. I tested SGD but saw that Adam generally led to higher validation accuracy.
- **Scheduler:** Implemented a StepLR scheduler (step_size=2, gamma=0.1) to help convergence by reducing the learning rate every two epochs.

Final Chosen Values:

- Epochs = 4
- Learning Rate = 0.00007
- Batch Size = 128
- Patience = 3 (early stopping)
- Scheduler = StepLR (step_size=2, gamma=0.1)

These values resulted in a balance between training speed, GPU utilization, and model performance.

Regularization Techniques

Techniques Used:

- **Early Stopping:** Patience set to 3 epochs, with a minimum improvement of 0.01. This technique helped prevent overfitting, saving the best model when validation accuracy stopped improving. It caused me a lot of trouble to set this up and sometimes it was easier to run the model until I saw a plateau in the validation loss or accuracy, and then run it again with reduced epochs as necessary.

- **Pre-trained Weights:** Using ResNet-50 pre-trained on ImageNet provided implicit regularization. This pretraining reduced the risk of overfitting by starting with learned features and requiring less epochs.
- **Learning Rate Scheduler:** Reducing the learning rate every 2 epochs helped to fine-tune the model. This helped my model avoid large weight updates that could disrupt its learned features.

Impact:

- Early stopping prevented overfitting while simultaneously saved time by avoiding excessive training/
- The learning rate scheduler allowed the model to refine its weights more effectively.

Data Augmentation Strategy

Techniques Used

Training Data:

- I Used RandomRotation and ColorJitter in my final model
- Normalize to match ImageNet statistics (mean=[0.485, 0.456, 0.406], std=[0.229, 0.224, 0.225]).
 - Generally I found that my models consistently worked better with less data augmentation. Perhaps I needed to train it for more epochs but didn't want to waste too much time training.

Test Data: Resize and normalize to match the training data.

Ablation Studies

- Initially experimented with additional augmentations like RandomFlip, AutoAugment, Crop, Resize, but found that adding too many distortions (especially on 32x32 images) decreased performance. I left these commented out in my part 2 code.

Results Analysis

- **Part 1: SimpleCNN:**
 - Test Accuracy: ~28%
 - Strengths: Simple and useful for learning CNN basics.
 - Weaknesses: Limited capacity to capture complex features, performance plateaued early.
 -
- **Part 2: Predefined - ResNet-50:**
 - Validation Accuracy: ~28%
 - Strengths: ResNet-50 is a much deeper network than my simple CNN. This enables the model to record more complex relationships with hierarchical features.
 - Weaknesses: ResNet is optimal for much larger images than those on the CIFAR dataset. Perhaps it was not the best architecture to use on this data as seen by the low validation accuracy.
- **Part 3: Pretrained - ResNet-50:**
 - Test Accuracy: ~58% on CIFAR-100.
 - OOD Score: ~40% on Kaggle test data.
 - Strengths: Significant performance improvement over SimpleCNN. Pre-trained weights helped the model generalize better.
 - Weaknesses: Resizing CIFAR-100 to 224x224 (ResNet's expected size) impacted performance on the Kaggle OOD test. This model was able to get up to 90% validation accuracy when both the test and training dataset were augmented to 224x224 however, I was unable to reproduce these results on the 32x32 CIFAR original dataset.

Experiment Tracking Summary

- Screenshots from WandB are included in the full experiment tracking section.
 - I added a CIFAR Test set score section to WandB as I noticed that this may be a better predictor of how the model would do in Kaggle than the validation accuracy.
- The experiments tracked included different hyperparameter configurations, learning rates, and augmentation strategies, ensuring thorough documentation of the model's performance and training process.
- In addition to WandB I initiated a git commit every time my model improved significantly and left a comment in the commit indicating which python script led directly to which WandB run.

- This was very helpful for when I got off track and my models were declining in accuracy so I could go back to an older commit and restore my best model.

dstnick Merge pull request #9 from dstnick/dev 23b24fd · 2 days ago 45 Commits		
.gitignore	initial commit	2 weeks ago
README.md	Update README.md	2 weeks ago
complex_cnn.py	graceful-donkey-53 from complex_cnn.py	3 days ago

Dev

#9 merged 2 days ago

wise-glitter-68 from effnet.py

#8 merged 2 days ago

descent-vortex-67 from effnet.py

#7 merged 3 days ago

dazzling-leaf-43 from simple_cnn.py

#6 merged 5 days ago

☐ ☐ Name (96 visualized)	num_w	ood_di	seed	epoch	lr	train_a	train_li	val_▼	val_los	cifar_sc	cifar_si
neat-planet-85	8	./data/ood	42	4	0	94.0875	1.02252	90.8	1.06839	-	-
decent-vortex-67	8	./data/ood	42	4	7.0000e-7	85.695	0.4763	82.4	0.60704	57.59	-
lilac-sky-79	8	./data/ood	42	4	7.0000e-7	83.2375	0.5728	80.86	0.71358	55.01	-
zesty-sky-66	8	./data/ood	42	10	7.0000e-10	79.9325	0.67159	73.65	0.91717	53.98	-
fresh-sun-65	8	./data/ood	42	1	0.00007	59.23	1.4937	69	1.08412	53.33	-
upbeat-surf-49	8	./data/ood	42	20	7.0000e-14	66.895	1.15922	62.77	1.30437	-	-
hearty-surf-88	8	./data/ood	42	4	0	63.4625	1.9355	60.87	1.97651	-	-
crisp-voice-56	8	./data/ood	42	20	7.0000e-14	89.17	0.36345	60.27	1.70341	-	-
dark-glitter-46	8	./data/ood	42	20	7.0000e-14	89.565	0.35285	60.25	1.70112	-	-
ancient-voice-54	8	./data/ood	42	4	0.000007	79.86	0.66864	60.15	1.4929	-	-

